ADAPTIVE GRAPH-RAG: MITIGATING REASONING COLLAPSE VIA SCORE-BASED PRUNING

Artur GARIPOV
Master in Data Sciences & Business Analytics (DSBA)
ESSEC Business School & CentraleSupelec
artur.garipov@essec.edu

Shan CAO
Master in Data Sciences & Business Analytics (DSBA)
ESSEC Business School & CentraleSupelec
b00827450@essec.edu

Code Repository:
https://github.com/Artur-Gar/graph-rag-benchmark

ABSTRACT

As Large Language Models (LLMs) scale, their ability to reason over graph-structured data remains limited. This project studies whether a Retrieval-Augmented Generation framework for graphs (Graph-RAG) improves pathfinding question answering compared with a full-graph baseline. We evaluate two tasks: predicting connectivity and extracting exact shortest paths between random node pairs. To avoid LLM-as-judge bias, we use deterministic Python-based evaluation, where the LLM is limited to structural extraction and response generation. Our experiments cover synthetic Erdős-Rényi graphs and sampled real subgraphs from the Cora and Facebook SNAP datasets, using three retrieval strategies: betweenness, path-aware, and degree-based scoring. The results show that Graph-RAG improves connectivity more consistently than exact path extraction, with stronger gains on larger or denser graphs and more mixed behavior on smaller synthetic settings. Overall, the study shows that context pruning can reduce reasoning failure and improve structural inference.

1 INTRODUCTION

Graphs are a natural way to represent complex systems such as social networks, citation networks, financial systems, and biological pathways. Traditionally, reasoning over graphs has relied on specialized algorithms such as Dijkstra's or A* Search, or on Graph Neural Networks (GNNs) trained with local message passing. Large Language Models (LLMs) offer a different setting: they can try to reason over graph structure directly from natural language prompts.

However, graph reasoning remains difficult for LLMs. A graph is naturally non-linear, but the model receives it as a linear token sequence. Prior work shows that performance depends strongly on how the graph is encoded and on the graph structure itself. Even when incident-style encoding makes neighborhood information easier to read, performance still drops as graph size grows. In dense graphs, many irrelevant edges can distract the model and hurt multi-hop reasoning.

Because transformer attention grows quadratically with sequence length, many irrelevant structural tokens can reduce the attention placed on the few nodes that matter for the target path. In a dense graph, the correct path may contain only a few edges, while the prompt may include hundreds of unrelated ones. This can lead to what we call "Reasoning Collapse" - a situation where the model fails to follow basic connectivity logic and hallucinates non-existent edges.

This project studies whether retrieving a focused subgraph with Graph-RAG can preserve or improve answer quality while reducing context complexity relative to a full-graph baseline. Using three retrieval strategies (`betweenness`, `path_aware`, and `degree`), we prune global graphs into query-centered subgraphs for shortest-path question answering.

Our core research questions are:

1. **Accuracy Improvement:** Does Graph-RAG improve connectivity detection and exact shortest-path extraction compared with full-context prompting?
2. **Scalability:** How does structural reasoning performance change as graph size and density increase?
3. **Heuristic Robustness:** Which retrieval strategy is the most reliable across different graph topologies, such as sparse citation networks and dense social networks?

2 Related Work

2.1 Large Language Models on Graphs

Recent studies have explored whether LLMs can act as graph reasoners. Fatemi et al. (2023) show that LLMs often perform better when graphs are represented with *Incident Encoding* rather than more rigid structural formats. Their evaluation, however, is focused mainly on small synthetic graphs.

2.2 Graph Neural Networks vs. LLMs

Traditional GNNs perform well on tasks such as node classification and link prediction through neighborhood aggregation. However, they require training data and may struggle on new reasoning tasks outside their training setting. LLMs offer a zero-shot alternative, but they do not have built-in biases for graph topology.

2.3 Retrieval-Augmented Generation for Structured Data

RAG has mainly been used for unstructured text. More recent work has adapted RAG to graph settings. In contrast to graph-level summarization, our work focuses on **path-centric micro-retrieval**: extracting a small but useful subgraph for a shortest-path reasoning query.

3 Problem Formulation

3.1 The Structural Question-Answering Task

Let $G = (V, E)$ be an undirected, unweighted graph where V is the set of nodes and E is the set of edges. Given a source node $s \in V$ and a target node $t \in V$, we define the task as a dual-objective problem:

1. **Connectivity Detection:** Predict whether there exists a valid path between s and t.
2. **Sequence Mapping:** If connected, extract the optimal sequence $P^* = (s, v_1, v_2, \ldots, t)$ such that $(v_i, v_{i+1}) \in E$ for all adjacent pairs and $|P^*|$ is minimal.

We define the **Reasoning Efficiency** (η) of an LLM as:

$$\eta = \frac{\mathbb{P}(\text{Valid Path} \mid G')}{|\text{Tokens}(G')|} \tag{1}$$

where $G' \subseteq G$ is the subgraph provided in the prompt. The goal is to keep the useful graph structure while removing unnecessary context.

3.2 Context Noise and Attention Dilution

In natural graphs such as Facebook, high-degree hubs can create a heavy prompt burden. If a hub node h with degree $d = 50$ is included through *Incident Encoding*, it may add 50 neighbor mentions to the prompt.

We describe this with the **Signal-to-Noise Ratio (SNR)** of a prompted graph context:

$$\text{SNR}(G') = \frac{|E_{\text{path}} \cap E'|}{|E' \setminus E_{\text{path}}|} \tag{2}$$

Including many irrelevant edges increases the denominator and lowers the useful signal. Since transformer self-attention scales as $O(L^2)$ with sequence length L, this creates two common failure modes:

- **Inference Latency (Timeouts):** Large prompts increase prefill time and can exceed timeout limits.
- **Reasoning Collapse:** The model spreads attention across too many irrelevant edges and loses focus on the important path structure.

The main challenge of Graph-RAG is therefore to balance noise reduction with pruning risk: too much context hurts reasoning, but too much pruning can remove useful edges.

4 METHODOLOGY

4.1 PHASE 1: STRUCTURAL DATA INDEXING

We preprocess G into an Incident Dictionary. We use Incident Encoding, since earlier work suggests that it is easier for LLMs to use than formats such as adjacency lists. We also use integer node names (e.g., "Node 0", "Node 1") rather than abstract labels, which makes the graph easier for the model to parse. For example: "Node 0 is connected to nodes 1, 2, and 3."

4.2 PHASE 2: ADAPTIVE SCORE-BASED RETRIEVAL

The retrieval mechanism selects a subset of nodes $V' \subset V$ to build the context subgraph $G' = G[V']$. To preserve routing structure while reducing prompt size, we use a multi-strategy heuristic expansion process.

If a path exists, we first compute the global shortest path P^* between source s and target t. Rather than expanding only from the endpoints, we initialize our Breadth-First Search (BFS) candidate set C and frontier using the full path P^*. We then expand the frontier for k hops, stopping early if the candidate set reaches an 80% capacity threshold ($0.8 \cdot N_{max}$).

Next, we rank candidate nodes with one of three scoring strategies:

- **Path-Aware:** Favors nodes that are close to both endpoints, $S(n) = -\big(d(s,n)+d(n,t)\big) + \lambda \cdot \text{Deg}(n)$, where $\lambda = 0.01$.
- **Betweenness:** Favors bridge-like nodes using subgraph betweenness centrality $C_B(n)$.
- **Degree:** Favors high-degree hub nodes, $S(n) = \text{Deg}(n)$.

Regardless of strategy, nodes on the core shortest path ($n \in P^*$) receive a maximum score so they are never pruned. The candidate set is then sorted by $S(n)$ in descending order, and the induced subgraph is built from the top-ranked nodes up to N_{max}.

4.3 PHASE 3: TASK GENERATION AND PROMPTING CONTRACT

To evaluate the framework, we generate trials by sampling random `(source, target)` pairs that are connected, and we compute the ground-truth shortest path with NetworkX.

To reduce free-form hallucinations, we use a strict Prompting Contract. The prompt requires JSON-only output and fixed endpoint constraints. The expected schema is:

```
{"is_connected": bool, "shortest_path": [int, ...] or null}
```

4.4 PHASE 4: DETERMINISTIC SCORING AND EXECUTION CONTROL

To avoid the subjectivity of LLM-as-judge evaluation, our pipeline uses deterministic Python-based scoring:

- **Connectivity Correctness:** Whether the predicted boolean matches the ground truth.

Algorithm 1 Adaptive k-hop Subgraph Retrieval

Require: Graph $G = (V, E)$, Source s, Target t, Max Hops k, Max Nodes N_{max}, Strategy $strat$
if not HasPath(G, s, t) **then**
  return $G[\{s, t\}]$
end if
$P^* \leftarrow$ ShortestPath(G, s, t)
$C \leftarrow P^*$
$F \leftarrow P^*$
for $i = 1$ **to** k **do**
  if $|C| \geq 0.8 \cdot N_{max}$ **then**
    break
  end if
  $F_{new} \leftarrow \bigcup_{v \in F}$ Neighbors(v, G)
  $F \leftarrow F_{new} \setminus C$
  $C \leftarrow C \cup F$
  if $F = \emptyset$ **then**
    break
  end if
end for
for each $n \in C$ **do**
  if $strat =$ "betweenness" **then**
    $S(n) \leftarrow C_B(n, G[C]) + (1.0$ if $n \in P^*$ else $0.0)$
  else if $strat =$ "path_aware" **then**
    $S(n) \leftarrow (10^6$ if $n \in P^*$ else $-(d(s, n) + d(n, t)) + 0.01 \cdot$ Deg$(n))$
  else
    $S(n) \leftarrow (10^6$ if $n \in P^*$ else Deg$(n))$
  end if
end for
$V' \leftarrow P^*$
$L \leftarrow$ Nodes in $C \setminus P^*$ sorted descending by $S(n)$
for each $n \in L$ **do**
  if $|V'| \geq N_{max}$ **then**
    break
  end if
  $V' \leftarrow V' \cup \{n\}$
end for
return $G' = G[V']$

- **Path Fidelity:** Whether the predicted path exactly matches a shortest valid path, including alternative shortest paths of equal optimal length.

Since large structural prompts create high latency, our `local_generate` function also includes exponential backoff and strict timeout handling.

5 Empirical Evaluation

5.1 Experimental Setup

We used a dual-track evaluation pipeline to test the structural reasoning limits of LLMs under different levels of graph complexity.

- **Datasets & Graph Synthesis:** Synthetic benchmarks were generated using Erdős-Rényi models for graph sizes $N \in \{20, 50, 100\}$. To test real-world generalization, we extracted connected sampled subgraphs from the **Cora** citation network and the **Facebook SNAP** dataset. In the real-data experiments, both sampled subgraphs contain 400 nodes.

- **The Prompting Contract:** We used a zero-shot JSON enforcement strategy, requiring the model to return a typed schema containing a connectivity boolean and a node list for the shortest path.
- **Metric Definitions:** System performance is measured using Connectivity Accuracy and Path Fidelity.
- **Inference Engine:** All trials were conducted locally on an NVIDIA RTX 4060 using LLaMA 3.1 (8B) via the Ollama API. To manage the high prefill latency of large graph prompts, we used a 180-second timeout with exponential backoff.

5.2 Results on Synthetic Topologies

Table 1 reports the average performance of the baseline and Graph-RAG across retrieval strategies.

Table 1: Average Performance on Synthetic Graphs (Averaged over Strategies)

	Connectivity Acc.		**Path Fidelity**	
Nodes (N)	Baseline	Graph-RAG	Baseline	Graph-RAG
20	93.3%	**96.7%**	**56.7%**	53.3%
50	93.3%	**100.0%**	36.7%	**43.3%**
100	83.3%	**90.0%**	10.0%	**26.7%**

Observation on Scaling Behavior: The synthetic results show a mixed but informative pattern. At $N = 20$, Graph-RAG gives only a small connectivity gain and a slight drop in average path fidelity relative to the baseline. This suggests that in smaller graphs, retrieval can sometimes remove useful local context without enough benefit from noise reduction. At $N = 50$, Graph-RAG improves both average connectivity and path fidelity. At $N = 100$, the benefit becomes much clearer: average path fidelity rises from 10.0% under the baseline to 26.7% under Graph-RAG. This suggests that subgraph retrieval becomes more useful as graph size grows and the full-context prompt becomes harder for the model to handle.

A strategy-level view supports the same pattern. At $N = 20$, `degree` performs worse than the baseline on path fidelity, while `betweenness` improves it and `path_aware` matches the baseline. At $N = 50$, `degree` and `path_aware` each improve path fidelity by 10 percentage points, while `betweenness` is tied with the baseline. At $N = 100$, all three strategies improve path fidelity, with `betweenness` and `degree` each gaining 20 percentage points and `path_aware` gaining 10.

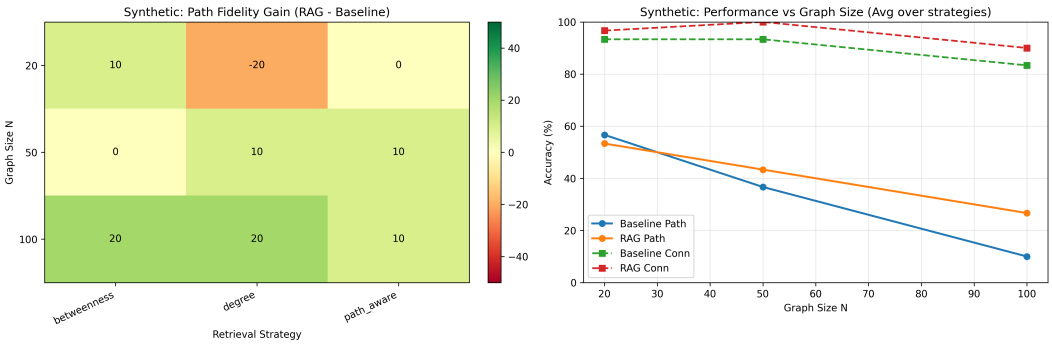

Figure 1: Synthetic graph results. The heatmap shows path fidelity gain (Graph-RAG minus baseline) across graph sizes and retrieval strategies. The line chart shows average baseline and Graph-RAG performance as graph size increases. The main pattern is that Graph-RAG becomes more helpful for path fidelity at larger graph sizes, especially at $N = 100$.

5.3 Results on Real-World Datasets

The real-data experiments were performed on sampled subgraphs with $N = 400$. The sampled Cora graph contains 778 edges, while the sampled Facebook graph contains 5506 edges, preserving the sparse-versus-dense contrast between the two datasets.

Table 2: System Performance on Real-World Sampled Subgraphs ($N = 400$)

Dataset	Strategy	Baseline Conn.	RAG Conn.	Baseline Fidelity	RAG Fidelity
Cora	betweenness	50.0%	90.0%	0.0%	10.0%
Cora	degree	50.0%	90.0%	0.0%	20.0%
Cora	path_aware	50.0%	90.0%	0.0%	10.0%
Facebook	betweenness	60.0%	70.0%	0.0%	10.0%
Facebook	degree	60.0%	100.0%	0.0%	10.0%
Facebook	path_aware	60.0%	90.0%	0.0%	20.0%

Averaged over strategies, Graph-RAG raises connectivity from 50.0% to 90.0% on Cora and from 60.0% to 86.7% on Facebook. For path fidelity, the baseline is 0.0% on both datasets, while Graph-RAG reaches 13.3% on average for both Cora and Facebook. These results show that Graph-RAG is clearly helpful for connectivity detection in real graphs, while the gain in exact shortest-path extraction is positive but still modest in absolute terms.

The real-data results show that Graph-RAG helps the model in both sparse and dense natural graphs, especially for connectivity detection. By reducing irrelevant structure in the prompt, retrieval allows the model to focus on a smaller set of informative edges and nodes. At the same time, exact multi-hop shortest-path reconstruction remains difficult, which suggests that pruning improves structural clarity more than algorithmic precision.

5.4 Ablation: Retrieval Strategy Performance

We compared the three ranking heuristics to determine which structural bias best supports LLM reasoning.

1. **Path-Aware (Distance-Centric):** On the real-data trade-off view, `path_aware` achieves an average connectivity gain of 35 percentage points and an average path gain of 15 percentage points. It performs especially well on Facebook path fidelity, where it reaches 20.0%. In the synthetic setting, it performs best at $N = 20$, ties at $N = 50$, and reaches 20.0% Graph-RAG path fidelity at $N = 100$.
2. **Degree (Hub-Centric):** `Degree` is the strongest overall strategy in the real-data experiments. It achieves the highest average connectivity gain (40 percentage points) and ties with `path_aware` for the highest average path gain (15 percentage points). It is also the only strategy that reaches 100.0% connectivity on Facebook. In the synthetic setting, `degree` becomes especially competitive at larger graph sizes, reaching 40.0% Graph-RAG path fidelity at $N = 100$.
3. **Betweenness Centrality:** `Betweenness` gives the weakest average real-data trade-off, with a 25 percentage point connectivity gain and a 10 percentage point path gain. Still, it remains competitive in some synthetic settings: it ties for the best Graph-RAG path fidelity at $N = 50$ and improves path fidelity strongly at $N = 100$. This suggests that betweenness is a reasonable middle-ground strategy, but not the most robust overall performer.

6 Discussion and Limitations

While Graph-RAG reduces context-induced hallucinations, it also creates Pruning Risk. In smaller or sparser graphs, retrieval may remove useful local context without enough benefit from noise reduction. This appears in the synthetic $N = 20$ setting, where Graph-RAG slightly underperforms the baseline on average path fidelity.

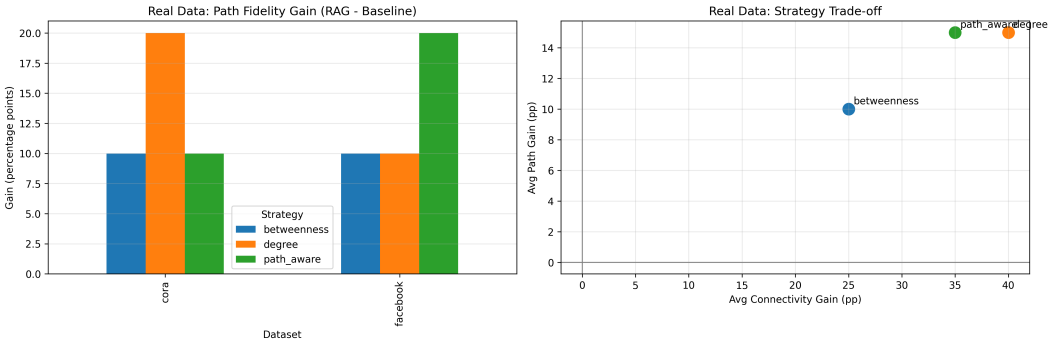

Figure 2: Real-data results. The bar chart shows path fidelity gain for each retrieval strategy on Cora and Facebook. The scatter plot shows the trade-off between average connectivity gain and average path gain across strategies. The main pattern is that `degree` gives the strongest average connectivity gain, while `degree` and `path_aware` are tied on average path gain and both outperform `betweenness`.

Our evaluation also shows a clear difference in task difficulty: connectivity detection is much easier for the LLM than exact shortest-path extraction. Even when Graph-RAG improves exact path recovery, path fidelity remains low in absolute terms on the sampled real-world datasets. This suggests that language models can often detect whether a route exists, but still struggle with the more exact step-by-step optimization needed to reconstruct the true shortest path.

Finally, strategy performance depends on graph topology. In the real-data setting, `degree` and `path_aware` outperform `betweenness` overall, but in synthetic graphs no single method dominates at every size. This shows that retrieval policy is itself an important modeling choice.

7 Conclusion and Future Work

Our results support the main conclusion of the project: Graph-RAG helps structural reasoning by reducing irrelevant context and improving connectivity prediction, especially as graph size or graph density increases. The strongest benefit appears in connectivity detection, while gains in exact shortest-path extraction are more limited.

On real-world sampled graphs, Graph-RAG raises average connectivity to 90.0% on Cora and 86.7% on Facebook, while average path fidelity reaches 13.3% on both datasets from a 0.0% baseline. On larger synthetic graphs, Graph-RAG also shows a clear advantage in path fidelity, especially at $N = 100$. These patterns are consistent with the view that context pruning becomes more useful as global graph noise increases.

Future work will explore Iterative Expansion methods. Rather than relying on a static one-pass retrieval step, an agentic Graph-RAG system could allow the LLM to request additional k-hop expansions when the initial subgraph is not enough. This may better balance pruning efficiency with path completeness.

Acknowledgments

This work was supported by the MLNS course at ESSEC Business School & CentraleSupélec.

References

- Fatemi, B., Halcrow, J., & Perozzi, B. (2023). Talk like a graph: Encoding graphs for large language models. *arXiv preprint arXiv:2310.04560*.
- He, X., et al. (2024). G-retriever: Retrieval-augmented generation for textual graph understanding. *Advances in Neural Information Processing Systems (NeurIPS)*.

- Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. *International Conference on Learning Representations (ICLR)*.